AI-Powered Educational Content Generation with Bloom's Taxonomy

A

REPORT

Submitted for Partial fulfillment of the Requirement
For the award of the degree of

BACHELOR OF TECHNOLOGY

In

Artificial Intelligence and Data Science

Submitted by

Shubham Umesh Parmar (124BTAI2003 - 62)
Tanishq Vijay Parmar (124BTAI2002 - 61)
Poonam Prasanna Thite (23UF17988AI056 - 51)

Under the guidance of

Mr. Pankaj Meheshre

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE
(An Autonomous Institute Affiliated to University of Mumbai)
MUMBAI - 400 088, MAHARASHTRA (INDIA)

2025-2026

Shah & Anchor Kutchhi Engineering College
(An Autonomous Institute Affilated to University of Mumbai)
Mumbai-400 088, MAHARASHTRA (India)

CERTIFICATE

It is my pleasure to certify that **Shubham Umesh Parmar** ,**Tanishq Vijay Parmar** , **Poonam Prasanna Thite** and worked under my supervision for the B. Tech. Project entitled **AI-Powered Educational Content Generation with Bloom's Taxonomy** and his/her work is of the level of requirement set up for the Project in **Artificial Intelligence and Data Science** by Shah & Anchor Kutchhi Engineering College, Mumbai.

Mr. Pankaj Meheshre
Assistant Professor
Department of Artificial Intelligence and Data Science
Shah & Anchor Kutchhi Engineering College
Mumbai – 400 088, India

The Oral and Practical examination of **Shubham Umesh Parmar** , **Tanishq Vijay Parmar** and **Poonam Prasanna Thite** , B. Tech. in Artificial Intelligence and Data Science, has been held on 08-11-2025.

External Examiner	**Internal Examiner**	**Guide Name**
Head of Department	**Principal**	College Seal

CANDIDATE'S DECLARATION

I hereby, declare that the work presented in the Project Report entitled **"AI-Powered Educational Content Generation with Bloom's Taxonomy"** for partial fulfillment of the requirement for the degree of B. Tech. in Artificial Intelligence and Data Science and submitted to the Department of Artificial Intelligence and Data Science at Shah & Anchor Kutchhi Engineering College, Mumbai, is an authentic record of my own work/cited work carried out during the period from July 2025 to November 2025 under the supervision of **Mr. Pankaj Meheshre**.

The matter presented in this Project Report has not been submitted elsewhere in part or fully to any other University or Institute for the award of any other degree.

Name of the Student	**Roll No.**	**Signature**
Shubham Umesh Parmar	124BTAI2003 - 62	
Tanishq Vijay Parmar	124BTAI2002 - 61	
Poonam Prasanna Thite	23UF17988AI056 - 51	

Date: 08-11-2025

Place: Mumbai

Shah & Anchor Kutchhi Engineering College
(An Autonomous Institute Affilated to University of Mumbai)
Mumbai-400 088, MAHARASHTRA (India)

CERTIFICATE OF PLAGIARISM CHECK

CERTIFICATE OF PLAGIARISM CHECK	
Name of the Student	Shubham Umesh Parmar , Tanishq Vijay Parmar , Poonam Prasanna Thite
Title of the Dissertation/Report	AI-Powered Educational Content Generation with Bloom's Taxonomy
Name of the Guide	Mr. Pankaj Meheshre
Name of the Department	Department of Artificial Intelligence and Data Science
Similar content (%) identified	13%
Acceptable Maximum Limit	10%
Name of the Similarity tool used for Plagiarism Report	Turnitin
Date of Verification	07-11-2025
Name & Sign of the Department Academic Integrity Panel (DAIP) Member	
Name & Sign of the Student	Shubham Umesh Parmar
	Tanishq Vijay Parmar
	Poonam Prasanna Thite
Name & Sign of the Guide	Mr. Pankaj Meheshre

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and sincere appreciation to our project guide, **Mr. Pankaj Meheshre**, (Assistant Professor) of the Department of Artificial Intelligence and Data Science, for his invaluable guidance, constant supervision, and unwavering support throughout this project. His insightful feedback and profound knowledge were instrumental in the successful completion of our work on "*AI-Powered Educational Content Generation with Bloom's Taxonomy*".

We extend our heartfelt thanks to Head of the Department of Artificial Intelligence and Data Science and Principal of **Shah & Anchor Kutchhi Engineering College**, for providing us with the necessary facilities and a conducive learning environment to complete our project successfully.

Our special thanks go to all the faculty members, lab assistants, and staff of the Department of Artificial Intelligence and Data Science for their cooperation and assistance during our course of study.

We are also indebted to our friends and classmates for their encouragement and insightful discussions, which helped us in refining our ideas.

Finally, we would like to express our profound gratitude to our families for their unwavering love, patience, and support, which has been our constant source of motivation.

Shubham Umesh Parmar
Tanishq Vijay Parmar
Poonam Prasanna Thite
Shah & Anchor Kutchhi Engineering College
Date: 08-11-2025

ABSTRACT

The manual creation of high-quality, diverse academic assessments and materials is a significant and time-consuming burden on educators. This project, "PaperGen," presents a secure, multi-user web application designed to automate and streamline this process. Leveraging the power of Google's Gemini Generative AI model, PaperGen allows educators to upload source material (text or PDF documents) and generate a wide variety of academic content. The system's key innovation is its dual-axis control, enabling users to precisely define content based on both Bloom's Taxonomy Level (e.g., Remembering, Applying, Creating) and Difficulty Level (Easy, Moderate, Challenging, Expert). PaperGen supports a comprehensive suite of content types, including Quizzes, Scenarios, Assignments, Presentation Topics, Mini Project Ideas, and Group Discussion Topics. It further enhances educator workflow through a two-stage generation process, allowing for the one-click creation of "Expected Answers" for questions and "Descriptions" for topics. The system is built on a robust backend using Python and Flask, with a secure SQLite database for user management and content history, and a responsive frontend built with HTML, TailwindCSS, and JavaScript. The result is an effective, efficient, and pedagogically-driven tool that empowers educators to focus more on teaching and less on content creation logistics.

Keywords: Generative AI, Educational Technology, Bloom's Taxonomy, Content Generation, Flask, Google Gemini, Web Application.

Contents

Certificate ii
Candidate's Declaration iii
Certificate of Plagiarism Check iv
Acknowledgement v
Abstract vi

1 Introduction **1**
1.1 Background 1
1.2 Motivation 2
1.3 Organization of the Report 2

2 Literature Review **3**
2.1 Survey of Existing system 3
2.2 Limitation of Existing system or research gap 3
2.3 Problem Statement and Objectives 4
2.4 Scope 5

3 Software Requirement Specification **6**
3.1 Introduction 6
3.2 Overall Description 6
3.3 External Interface Requirements 7
3.4 System Features 8
3.4.1 Feature 1: User Authentication 8
3.4.2 Feature 2: Primary Content Generation 9
3.4.3 Feature 3: Secondary Content Generation 9
3.5 Other Nonfunctional Requirements 10

4 Project Scheduling and Planning **11**
4.1 Work Breakdown Structure 11

5 Proposed System **13**
5.1 Feasibility Study 13
5.2 Analysis and Framework 13
5.3 Details of Hardware & Software 14
5.4 Design Details 15

5.5 Methodology 16

6 Implementation Plan for Next Semester **17**

6.1 Direct Document Export 17

6.2 Enhanced Content Support 17

6.3 Custom Prompt Templates 17

7 Summary **18**

Bibliography **19**

Appendices **20**

A Appendix **21**

A.1 Plagiarism Report 21

Chapter 1

Introduction

This chapter presents a comprehensive overview of the 'PaperGen' project. The primary focus of this project is to address a critical pain point in the modern educational sector: the intensive manual labor required for academic content creation. We introduce PaperGen as a novel, web-based application specifically engineered to serve as an intelligent assistant for educators. The chapter begins by establishing the background and context of this problem, detailing the significant challenges educators face in consistently producing diverse, high-quality, and pedagogically-sound materials such as quizzes, assignments, and case studies. It then explores the core motivation for the project, which stems from the need to harness the power of advanced Generative AI to automate these tasks, thereby freeing educators to focus on direct instruction and student engagement. Finally, this chapter concludes by outlining the organization of the entire report, providing a clear roadmap for the subsequent sections which detail the project's literature review, system requirements, proposed architecture, implementation, and future scope.

1.1 Background

In the modern educational landscape, the demand for high-quality, diverse, and tailored learning materials is greater than ever. Educators are expected to produce a constant stream of content, including quizzes, assignments, case studies, and project ideas, often customized to different learning levels. This manual process is incredibly time-consuming, repeti-

tive, and detracts from the time educators can spend on direct instruction and student interaction. With the recent advancements in Large Language Models (LLMs), the potential for Generative AI to alleviate this burden has become clear. These models can understand context from source materials (like lecture notes or textbooks) and generate new, relevant content.

1.2 Motivation

The primary motivation for "PaperGen" was to bridge the gap between the raw power of Generative AI and the specific, nuanced needs of an educator. While many tools can generate a quiz, they often lack the fine-grained pedagogical control required for a structured curriculum. An educator doesn't just need "a question"; they need an "Applying-level question of Moderate difficulty" or a "Creating-level project idea that is Challenging." Our goal was to create a tool that moves beyond generic content generation to become a genuine assistant for educators.

1.3 Organization of the Report

- **Chapter 2:** presents a literature review of existing systems and defines the project's problem statement, objectives, and scope.
- **Chapter 3:** provides the Software Requirement Specification (SRS), detailing functional and nonfunctional requirements.
- **Chapter 4:** outlines the project's scheduling and planning.
- **Chapter 5:** describes the architecture and methodology of the proposed system, including the core prompt engineering framework.
- **Chapter 6:** discusses the implementation and plans for future work.
- **Chapter 7:** concludes the report with a summary of the project's achievements.

Chapter 2

Literature Review

2.1 Survey of Existing system

Our research surveyed three main categories of existing tools:

1. **General-Purpose LLMs (e.g., ChatGPT, Google Gemini):** These models are extremely powerful and flexible. An educator can manually write a complex prompt to generate a quiz. However, this requires significant prompt engineering expertise from the end-user, and the results can be inconsistent.

2. **Quiz-Maker Tools (e.g., Kahoot!, Quizlet):** These platforms are excellent for creating and delivering gamified quizzes. However, their focus is on the delivery and "flashcard" aspect, not on the initial *generation* of deep, analytical questions from source text. The content creation is still largely manual.

3. **Basic Content Generators:** Several online tools offer "Quiz from Text" functionality. These are often a thin wrapper around an API and suffer from the same lack of pedagogical control as general-purpose LLMs. They typically only create simple, fact-recall (Remembering-level) questions.

2.2 Limitation of Existing system or research gap

The key research gap identified is the lack of a **multi-axis control system** for educational content generation. No existing tool, to our knowledge,

effectively combines:

- **Diverse Content Types:** (Quiz, Scenario, Project Ideas, etc.) in one interface.
- **Cognitive Skill Control:** Explicitly targeting levels of Bloom's Taxonomy.
- **Difficulty Control:** A separate, independent axis for adjusting question complexity.
- **Integrated Workflow:** A system that not only generates questions but also understands the logical next step, such as generating the corresponding answers or descriptions.

2.3 Problem Statement and Objectives

Problem Statement:

Educators lack an efficient, secure, and integrated tool that can generate a wide variety of academic materials from a single source text, while providing precise control over the content's cognitive skill level (Bloom's Taxonomy) and difficulty.

Objectives:

1. To develop a secure, multi-user web application using Flask, with a persistent SQLite database for user management and content history.
2. To implement a full user authentication system, including registration, login, and mandatory email verification using secure tokens.
3. To integrate Google's Gemini Generative AI API as the core engine for content generation.
4. To design and implement a "Dual-Axis" control system, allowing users to select both a **Bloom's Level** and a **Difficulty Level**.

5. To support a diverse suite of content types: Quiz, Scenario, Assignment, Presentation, Mini Project, and Group Discussion.

6. To create a two-stage, context-aware workflow where the UI offers secondary actions (e.g., "Generate Expected Answers," "Generate Description") based on the primary content generated.

7. To apply robust backend cleaning and formatting to all AI output, ensuring it is clean, readable, and free of unwanted artifacts (like markdown tables or symbols).

2.4 Scope

The system is a **web-based application** accessible from any modern browser. It is intended for use by educators (teachers, university professors, corporate trainers). The scope includes processing user-provided text (pasted or uploaded as `.pdf` and `.txt`) and generating new text-based content. The scope *does not* include:

- Grading or processing student submissions.
- Generating non-text content like images or videos.
- Hosting or delivering the generated content (e.g., as a live quiz platform).

Chapter 3

Software Requirement Specification

3.1 Introduction

This section provides the Software Requirement Specification for the PaperGen system.

- **Purpose:** The purpose of this product is to provide an automated, AI-powered tool for educators to generate high-quality, pedagogically-sound academic content.
- **Document Conventions:** This document follows a standard SRS structure.
- **Intended Audience:** The primary audience includes the project developers, the project guide (for evaluation), and future developers who may maintain or extend the system.

3.2 Overall Description

- **Product Perspective:** PaperGen is a new, self-contained, multi-user web application. It is accessed via a web browser and relies on an external software interface (Google's Gemini API) to function.
- **Product Functions:**
  - User Account Management (Register, Login, Logout, Email Verification).
  - Secure, session-based access to user-specific data.

  - Text and File (PDF, TXT) processing as a context source.
  - A primary generation interface with controls for Bloom's Level, Difficulty, and Content Type.
  - A chat-style message display for the generation history.
  - A secondary, context-aware generation system (Answers/Descriptions).
  - A dashboard for viewing and deleting past generations.

- **User Classes:** "Educator" (Authenticated User). This is the primary user class with access to all content generation features and their own history.

- **Operating Environment:**
  - **Client:** Any modern web browser (e.g., Chrome, Firefox, Safari) on any OS (Windows, macOS, Linux, Android, iOS).
  - **Server:** The application is built to run on a standard Python web server (e.g., Gunicorn) on a Linux-based cloud platform.

- **Design and Implementation Constraints:**
  - The system *must* have a persistent internet connection to call the Google Gemini API.
  - All user passwords *must* be securely hashed and never stored in plain text.
  - The system is built using the Flask web framework for Python.
  - The database *must* be SQLite for portability and ease of setup.

3.3 External Interface Requirements

- **User Interfaces:** The UI is a clean, single-page application (SPA) style interface rendered by Flask templates. It features:
  1. A persistent left sidebar for navigation and generation history.
  2. A main chat-style content area.

3. A bottom input form with centered dropdowns for "Bloom's Level" and "Difficulty," a text/upload bar, and centered buttons for "Content Type."
4. Contextual buttons (Copy, Download, Generate Answers) that appear on AI-generated messages.

- **Hardware Interfaces:** None.
- **Software Interfaces:**
  1. **Google Gemini API:** The system communicates with the Google API endpoint via HTTPS POST requests, sending a JSON payload containing the dynamically constructed prompt and receiving a JSON response.
  2. **Flask-Mail (SMTP):** Uses an SMTP server (e.g., Gmail) to send verification emails to new users.
- **Communications Interfaces:** HTTPS protocol for all client-server and server-API communication.

3.4 System Features

3.4.1 Feature 1: User Authentication

- *Description:* Full user management system.
- *Functional Requirements:*
  - REQ-1.1: A user shall be able to register with a username, email, and password.
  - REQ-1.2: The system shall hash the password before saving it to the database.
  - REQ-1.3: The system shall send a unique, timed verification link to the user's email.

- REQ-1.4: A user shall not be able to log in until their email is verified.
- REQ-1.5: A user shall be able to log in with their email and password.
- REQ-1.6: The system shall maintain a persistent session for logged-in users.

3.4.2 Feature 2: Primary Content Generation

- *Description:* The core generation workflow.
- *Functional Requirements:*
  - REQ-2.1: The user shall be able to paste text into a textarea.
  - REQ-2.2: The user shall be able to upload a `.pdf` or `.txt` file.
  - REQ-2.3: The system shall extract text from the uploaded file.
  - REQ-2.4: The user must select one "Bloom's Level" (e.g., "Apply").
  - REQ-2.5: The user must select one "Difficulty Level" (e.g., "Moderate").
  - REQ-2.6: The user must select one "Content Type" (e.g., "Scenario").
  - REQ-2.7: On submit, the system shall construct a dynamic prompt based on all selections and the source text.

3.4.3 Feature 3: Secondary Content Generation

- *Description:* Context-aware generation of supporting materials.
- *Functional Requirements:*
  - REQ-3.1: If the primary content is a "Quiz," "Scenario," or "Assignment," the system shall display a "Generate Expected Answers" button.

- REQ-3.2: If the primary content is a "Presentation," "Mini Project," or "Group Discussion," the system shall display a "Generate Description" button.
- REQ-3.3: Clicking this button shall send the primary content back to the AI with a new prompt (e.g., "Generate answers for these questions") and display the result.

3.5 Other Nonfunctional Requirements

- **Performance:**
  - Web pages shall load in under 3 seconds.
  - AI generation requests should ideally complete within 10-15 seconds.
- **Security:**
  - All user-specific routes must be protected by the @login-required decorator.
  - The system shall be protected against Cross-Site Scripting (XSS) by using Jinja2 templating, which auto-escapes output.
- **Usability:**
  - The interface shall be responsive and usable on mobile devices and tablets.
  - The workflow shall be intuitive, with clear labels and immediate visual feedback (e.g., loading indicators).
- **Reliability:** The system must gracefully handle API errors (e.g., from Gemini) and display a user-friendly error message instead of crashing.

Chapter 4

Project Scheduling and Planning

4.1 Work Breakdown Structure

The project was executed over a 14-week period, broken down as follows:

- **Weeks 1-2: Foundation & Planning**
  - Finalized project scope, objectives, and technology stack (Python, Flask, SQLite).
  - Set up the virtual environment, Flask application structure, and Git repository.
  - Designed the database schema (User, GeneratedContent).
- **Weeks 3-4: Core User System**
  - Implemented database models using Flask-SQLAlchemy.
  - Built the user registration, login, and logout functionality.
  - Implemented password hashing (`werkzeug.security`).
  - Integrated Flask-Mail and `itsdangerous` for secure, timed email verification.
- **Weeks 5-6: Basic Generation Interface**
  - Developed the main `index.html` template with TailwindCSS.
  - Created the basic chat interface layout.
  - Implemented the first Gemini API call with a static prompt for "Quiz" generation.

  - Wrote file-handling logic for `.txt` and `.pdf` (PyMuPDF) uploads.

- **Weeks 7-8: Advanced Control System**
  - Implemented the "Bloom's Level" and "Difficulty Level" dropdowns.
  - Created the dynamic prompt engineering framework in `app.py` to inject instructions based on these controls.
  - Expanded prompts to support all six content types (Scenario, Presentation, etc.).

- **Weeks 9-10: Two-Stage Workflow**
  - Developed the "Generate Expected Answers" (`/generate_answers`) endpoint and prompt.
  - Developed the "Generate Description" (`/generate_description`) endpoint and prompt.
  - Wrote client-side JavaScript to dynamically show the correct button based on content type.

- **Weeks 11-12: Features & Polishing**
  - Implemented the generation history feature, linking the table to `current_user`.
  - Built the "Dashboard" page for viewing and deleting past content.
  - Developed and iteratively refined the aggressive regex cleaning functions to remove markdown tables and inconsistent spacing.

- **Weeks 13-14: Final Testing & Reporting**
  - Conducted end-to-end testing of all features.
  - Performed UI testing on different browsers and screen sizes.
  - Finalized all code, removed bugs, and prepared this project report.

Chapter 5

Proposed System

5.1 Feasibility Study

- **Technical Feasibility:** The project is highly feasible. The chosen technologies (Python, Flask) are mature, well-documented, and suited for web application development. The Google Gemini API is a stable, publicly available service.
- **Economic Feasibility:** The project has very low economic overhead. All core technologies (Python, Flask, SQLite) are open-source and free. The Gemini API offers a generous free tier sufficient for development and testing, with scalable, low-cost pricing for production.
- **Operational Feasibility:** The system is operationally feasible. As a web-based application, it requires no installation by the end-user. The UI is designed to be simple and intuitive, requiring minimal training for an educator familiar with web tools.

5.2 Analysis and Framework

The core of PaperGen is its **Dynamic Prompt Engineering and Cleaning Framework**.

1. **Collection:** The system collects four inputs: Source Text, Bloom's Level, Difficulty Level, and Content Type.
2. **Dynamic Construction:** Instead of using a single, massive prompt, the backend (`app.py`) dynamically constructs a precise prompt. It

retrieves instructional "blocks" from dictionaries (e.g., `bloom_data`, `difficulty_data`) and combines them with formatting rules specific to the chosen content type.

3. **Generation:** This composite prompt is sent to the Google Gemini API.

4. **Aggressive Cleaning:** The raw text response from the AI is processed through a multi-step regex cleaning pipeline. This pipeline is crucial and performs several tasks:

   - Removes unwanted characters (e.g., *, ** M).
   - Strips all markdown table syntax (e.g., —...— or ———).
   - Removes markdown headings (e.g., #).
   - Strips all leading whitespace from every line to fix indentation.
   - Ensures a newline follows labels (e.g., Answer:, Description:).
   - Forces a single blank line (e.g., \n\n) before each new numbered item (e.g., Q1., 1.).
   - Consolidates any remaining 3+ newlines down to 2.

5. **Display:** The final, clean, and well-formatted text is sent to the user's interface.

5.3 Details of Hardware & Software

- **Software (Development):**
  - **OS:** Windows/macOS
  - **Language:** Python 3.10+
  - **Framework:** Flask
  - **Database:** SQLite 3
  - **Libraries:** `Flask-SQLAlchemy`, `Flask-Login`, `Flask-Mail`, `PyMuPDF`, `google-generativeai`, `python-dotenv`

  - **Frontend:** HTML5, TailwindCSS, JavaScript (ES6)
  - **VCS:** Git

- **Software (Deployment):**
  - **Server:** Any WSGI-compliant server (e.g., Gunicorn)
  - **Host:** Any cloud platform (e.g., PythonAnywhere, Heroku, AWS)
  - **Client:** Modern Web Browser

- **Hardware:**
  - **Development:** Standard PC/Laptop (No special requirements).
  - **Deployment:** Standard cloud VM instance.
  - **Client:** Any device with a web browser (PC, tablet, smartphone).

5.4 Design Details

The UI/UX design is centered on simplicity and efficiency, resembling a "tool" more than a "social app."

- **Layout:** A three-column layout is used: a persistent sidebar for navigation/history, a main content area for the chat, and the input form.
- **Input Form:** The form is designed to be the "control panel." We intentionally centered the "Bloom's Level" and "Difficulty" dropdowns *above* the text input to encourage users to set these parameters first. The "Content Type" buttons are placed *below*, as they are the final "action" to select.
- **Chat Interface:** A familiar chat bubble interface is used to display the user's request and the AI's response, making the history easy to follow.

- **Contextual Actions:** Instead of cluttering the UI, actions (Copy, Download, Generate Answers) only appear on the AI's messages, providing a clean and context-sensitive experience.

5.5 Methodology

The project followed an **iterative, feedback-driven methodology**.

1. **Phase 1 (Core Functionality):** Built the user system and a single-prompt "Quiz" generator.
2. **Phase 2 (Expansion):** Added the Bloom's/Difficulty controls and all other content types.
3. **Phase 3 (Refinement):** Implemented the two-stage generation (Answers/Descriptions) based on the "need" for a complete workflow.
4. **Phase 4 (Cleaning):** This phase was iterative throughout. After each feature addition, we tested the AI's output and continuously refined the regex cleaning functions to handle unwanted formatting (tables, spacing, symbols), which proved to be a critical part of the project.

Chapter 6

Implementation Plan for Next Semester

6.1 Direct Document Export

A key planned feature is the ability to export the generated content directly into common document formats.

- `.txt`: A simple "Download" button is already implemented.
- `.pdf` / `.docx`: Future work will involve integrating libraries (e.g., `FPDF` or `python-docx`) to generate professionally formatted documents (including headers, footers, and proper page breaks) from the cleaned text, making them "print-ready" for educators.

6.2 Enhanced Content Support

- **Image & Diagram Generation:** Integrate an image generation model to create or suggest simple diagrams, charts, or relevant images to accompany the generated text, especially for technical or scientific subjects.
- **Rubric Generation:** Add a new content type to generate a "Grading Rubric" based on a "Mini Project" or "Assignment" question.

6.3 Custom Prompt Templates

Allow educators to save their "perfect" combination of Bloom's Level, Difficulty, and custom instructions as a template for quick reuse.

Chapter 7

Summary

This project successfully achieved its objective of creating "PaperGen," a secure, multi-user web application for intelligent academic content generation. By integrating Google's Gemini API with a novel dual-axis control system (Bloom's Taxonomy and Difficulty Level), the application provides educators with an unprecedented level of control and versatility.

The system supports six distinct content types and features an innovative two-stage workflow for generating secondary materials like answers and descriptions. A significant part of the project involved developing a robust, multi-step cleaning pipeline to transform raw AI output into clean, well-formatted, and professional text suitable for academic use.

PaperGen stands as a successful proof-of-concept, demonstrating how guided Generative AI can be harnessed as a powerful and practical tool to support educators, automate repetitive tasks, and allow them to focus on what matters most: teaching.

Bibliography

[1] K. P. Karri et al., "Automated Question Paper Generator Using LLM," *International Journal of Research and Innovation in Applied Science (IJRIAS)*, vol. X, no. IV, 2025.

[2] R. Rodriguez-Torrealba, E. Garcia-Lopez, and A. Garcia-Cabot, "End-to-End generation of Multiple-Choice questions using Text-to-Text transfer Transformer models," *Expert Systems With Applications*, vol. 208, p. 118258, 2022.

[3] K. Hwang, S. Challagundla, M. M. Alomair, L. K. Chen, and F. Choa, "Towards AI-Assisted Multiple Choice Question Generation and Quality Evaluation at Scale: Aligning with Bloom's Taxonomy," in *NeurIPS 2023 Workshop on Generative AI for Education (GAIED)*, 2023.

[4] S. Jena, A. Raina, H. Raina, and C. Nehete, "AI based Dynamic Model for Question Paper Generation and Performance Monitoring," in *2022 International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*, 2022.

[5] P. Thotad, S. Kallur, and S. Amminabhavi, "Automatic Question Generator Using Natural Language Processing," *Journal of Pharmaceutical Negative Results*, vol. 13, no. S10, pp. 2759-2764, 2022.

[6] K. B. Dhomse and I. Ranjan, "Automatic Question Paper Generation using ML: A Review," *Turkish Journal of Computer and Mathematics Education*, vol. 12, no. 1S, pp. 239-245, 2021.

[7] Google. (2024). *Gemini: A Family of Highly Capable Multimodal Models.* Retrieved from https://ai.google/discover/gemini/

[8] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python.* O'Reilly Media.

Appendices

Appendix A

Appendix

A.1 Plagiarism Report

Tanishq Parmar - 124BTAI2002
Shubham Parmar - 124BTAI2003
Poonam Thite - 23UF17988AI056

Papergen_Report.pdf

SAKEC - Shah and Anchor Kutchhi Engineering College

Document Details

Submission ID
trn:oid:::3618:120205731

Submission Date
Nov 7, 2025, 2:02 PM GMT+5:30

Download Date
Nov 7, 2025, 2:06 PM GMT+5:30

File Name
Papergen_Report.pdf

File Size
166.9 KB

29 Pages

4,435 Words

26,056 Characters

13%Overall Similarity

Thecombinedtotalofallmatches,includingoverlappingsources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 10 words)

Match Groups

25Not Cited or Quoted13%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations0%
Matches that are still very similar to source material

0 Missing Citation0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
1% Publications
10% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags forReview

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.
A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 25Not Cited or Quoted13%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations0%
  Matches that are still very similar to source material
- 0 Missing Citation0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted0%
  Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
1% Publications
10% Submitted works (Student Papers)

Top Sources

Thesourceswiththe highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.coursehero.com	8%
2	Submitted works	mctrajiv on 2024-04-29	<1%
3	Internet	grotrian.nsu.ru	<1%
4	Submitted works	Arab Open University on 2025-05-02	<1%
5	Submitted works	University of Nottingham on 2023-05-11	<1%
6	Submitted works	APJ Abdul Kalam Technological University, Thiruvananthapuram on 2025-03-23	<1%
7	Submitted works	University of Technology, Sydney on 2015-12-07	<1%
8	Internet	www.slideshare.net	<1%
9	Submitted works	University of Birmingham on 2023-09-17	<1%
10	Submitted works	University of Strathclyde on 2020-04-06	<1%

11 Internet

earthquake.usgs.gov <1%

12 Submitted works

mctrajiv on 2025-04-23 <1%

13 Submitted works

mctrajiv on 2024-05-01 <1%

14 Submitted works

Manchester Metropolitan University on 2024-09-26 <1%

15 Submitted works

University of Maryland, University College on 2014-10-06 <1%

16 Submitted works

University of Maryland, University College on 2019-07-15 <1%